

Authentication & User Profile System Architecture

This document describes the conceptual and technical implementation of the Login, Registration, Forgot Password, and Profile Management systems for the **Tos Tov** project.

1. Core Concepts

A. Authentication Flow (Supabase + NestJS)

We use **Supabase Auth** as the primary identity provider. It handles user registration, email verification, and session management. **NestJS** acts as the backend orchestrator, managing additional business logic and profile data.

B. Profile Management

While Supabase handles *Identity* (email/password), the **profiles** table in our database handles *User Data* (bio, avatar, preferences). This separation ensures security while allowing flexibility in user data management.

2. Feature Workflows

Registration

1. **Frontend:** User submits **email** and **password**.
2. **Backend (NestJS/Supabase):**
 - Supabase creates a new entry in **auth.users**.
 - A database trigger (or service call) creates a corresponding entry in the **public.profiles** table.
3. **Security:** Emails are verified via a confirmation link before the account is fully active.

Login

1. **Frontend:** User submits credentials.
2. **Backend:** Supabase validates credentials and returns a **JWT (JSON Web Token)**.
3. **Storage:** The frontend stores the token (usually in **localStorage** or a Secure Cookie) and includes it in the **Authorization** header for subsequent API calls.

Forgot Password

1. **Request:** User provides their email.
 2. **Link Generation:** Supabase sends a secure, time-limited "Magic Link" or "Recovery Token" to the user's email.
 3. **Reset:** User clicks the link, which redirects to the frontend with a temporary session allowed only for updating the password.
-

3. API Communication (Frontend & Backend)

The communication follows a RESTful pattern.

Feature	Method	Endpoint	Description
Register	POST	/auth/register	Create a new user account
Login	POST	/auth/login	Authenticate and receive JWT
Reset Password	POST	/auth/reset-password	Request a recovery email
Get Profile	GET	/profiles/me	Fetch current user's details
Update Profile	PATCH	/profiles/update	Modify bio, avatar, etc.

🔗 Real Data Example (Login Request/Response)

Request (Frontend -> Backend):

```
{
  "email": "traveler@example.com",
  "password": "SecurePassword123!"
}
```

Response (Backend -> Frontend):

```
{
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "traveler@example.com",
    "last_login": "2026-06-17T10:00:00Z"
  },
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 3600
}
```

4. Security Concepts

🛡️ 1. JWT (JSON Web Tokens)

Instead of session IDs stored on the server, we use JWTs.

- **Stateless:** The server doesn't need to store a list of logged-in users.
- **Verification:** The backend validates the signature of the token to ensure it hasn't been tampered with.

🛡️ 2. Hashing & Salting (Handled by Supabase)

- **Hashing:** Passwords are never stored as plain text. They are converted into a cryptographic string (e.g., using `bcrypt`).
- **Salting:** A random string (salt) is added to each password before hashing to prevent "Rainbow Table" attacks (where attackers pre-calculate hashes for common passwords).

3. RLS (Row Level Security)

Our Supabase PostgreSQL database uses RLS.

- **Concept:** Even if an attacker gets access to the API, they can only read/write data that "belongs" to them.
- **Example Policy:** `ALTER POLICY "Users can update own profile" ON profiles FOR UPDATE USING (auth.uid() = id);`

4. CORS (Cross-Origin Resource Sharing)

The backend only accepts requests from trusted domains (e.g., your Vue.js production URL). This prevents malicious sites from making requests on behalf of your users.

5. Input Validation (DTOs)

In NestJS, we use **Data Transfer Objects (DTOs)** and `class-validator` to ensure that data sent by the user is clean.

- *Example:* If a user tries to send a 5,000-character bio, the DTO will reject it before it touches the database.

5. Important Implementation Code

If your teacher asks "How does it actually work in code?", these are the three most important parts to show:

A. Route Protection (NestJS `AuthGuard`)

This code intercepts every request to see if the user has a valid token.

```
// apps/api/src/common/guards/auth.guard.ts
@Injectable()
export class AuthGuard implements CanActivate {
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    // 1. Get token from header: "Authorization: Bearer <TOKEN>"
    const token = request.headers['authorization']?.replace('Bearer ', '');

    if (!token) return false;

    // 2. Ask Supabase: "Is this token valid?"
    const { data, error } = await
this.supabaseService.anonClient.auth.getUser(token);

    if (error) return false;

    // 3. Attach user data to the request object for use in controllers
    request.user = data.user;
    return true;
  }
}
```

```
}  
}
```

B. Database Privacy (PostgreSQL RLS)

Even if someone bypasses the API, the database itself blocks unauthorized access using Row Level Security.

```
-- packages/supabase/supabase/migrations/..._profiles.sql  
  
-- Enable the firewall (RLS)  
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;  
  
-- Rule: Users can only INSERT a profile if the ID matches their logged-in ID  
CREATE POLICY "Allow authenticated insert on profiles"  
ON profiles FOR INSERT TO authenticated  
WITH CHECK (auth.uid() = id);  
  
-- Rule: Users can view all profiles (to find friends)  
CREATE POLICY "Allow authenticated select on profiles"  
ON profiles FOR SELECT TO authenticated  
USING (true);
```

C. Profile Management (NestJS Service)

This shows how we update profile data securely.

```
// apps/api/src/modules/profiles/profiles.service.ts  
async updateById(id: string, dto: UpdateProfileDto) {  
  // We use the adminClient to ensure the system can update fields,  
  // but the ID is passed from the AuthGuard (the logged-in user).  
  const { data, error } = await this.supabaseService.adminClient  
    .from('profiles')  
    .update({  
      first_name: dto.first_name,  
      last_name: dto.last_name,  
      bio: dto.bio  
    })  
    .eq('id', id) // Security: Ensure we only update the current user's record  
    .select()  
    .single();  
  
  if (error) throw new InternalServerErrorException(error.message);  
  return data;  
}
```

D. Password Recovery (Forgot Password)

We use a passwordless OTP (One-Time Password) flow for recovery.

```
// apps/api/src/modules/auth/auth.service.ts
async forgotPassword(email: string) {
  // Supabase sends a secure OTP to the user's email
  const { error } = await this.client.auth.signInWithOtp({
    email,
    options: {
      shouldCreateUser: false, // Security: Don't create new accounts here
    },
  });

  if (error) throw new BadRequestException(error.message);
  return { msg: 'OTP sent' };
}
```

6. Security Summary Checklist

- ☒ Use HTTPS for all communications.
- ☒ Store passwords using strong hashing (Argon2/Bcrypt).
- ☒ Implement Rate Limiting to prevent brute-force attacks on login.
- ☒ Sanitize all user inputs to prevent SQL Injection and XSS.
- ☒ Apply RLS policies to sensitive database tables.